State Management, Part 1

Debugging, Stateless vs Stateful, and where state should live

Dinu-Ștefan Rusu
FILS, Universitatea Politehnica București · Autumn 2026

TODAY

What you should leave with

Debug on purpose

Launch and attach DevTools, read the inspector, set breakpoints and step through Dart.

The three trees

Widget, Element and RenderObject, and what setState actually triggers.

Stateless vs Stateful

When a widget really needs a State object, and when it genuinely does not.

Sharing state

Lifting state up, InheritedWidget, and why value equality decides rebuilds.

PART 1 · DEBUGGING

Seeing what your app is actually doing

DevTools, breakpoints, and useful logging

The debugging toolbox

Hot reload

Injects changed source into the running VM and rebuilds. State objects survive.

Hot restart

Rebuilds from scratch and drops all state. Use it whenever initState or a global changed.

debugPrint & assert

Cheap, always available, and compiled out of release builds entirely.

Breakpoints

Stop on a line, inspect every variable in scope, step through the frame.

DevTools

Inspector, performance, CPU, memory, network: a browser tab attached to your app.

Read the error

Flutter's exception boxes name the widget and the constraint that failed.

print, debugPrint, and assert

```
print('value: $x');        // logcat truncates
debugPrint('value: $x');   // throttled, lossless
assert(count >= 0, 'count negative');

debugDumpApp();             // widget tree
debugDumpRenderTree();      // sizes + constraints
debugPrintRebuildDirtyWidgets = true;
```

Why not just print?
Android's logcat drops long lines. `debugPrint` throttles output so nothing disappears.
`assert` takes a message. Use it for invariants you want to hear about in the lab.

Logging is a primary debugging tool. It is the only tool that works identically on a simulator, a real phone and a colleague's machine.

STATE

Pick the smallest tool that works

TOOL	SCOPE	BOILERPLATE	TESTABLE	USE WHEN
setState	one widget	none	partly	the state never leaves the screen
Lifting up	a subtree	low	yes	two siblings share one value
InheritedWidget	any descendant	medium	yes	a value is read far below its owner
Provider	any descendant	low	yes	the same, with less ceremony
BLoC / Cubit	whole feature	high	excellent	events and side effects

Provider wraps InheritedWidget. Riverpod is its successor, without the BuildContext.

This is Part 1. setState, lifting up and InheritedWidget carry most screens. The last column is Lecture 6.

NUMBERS

Why the frame budget matters

16.7 ms

per frame at 60 Hz

8.3 ms

per frame at 120 Hz, most 2026 phones

1

missed frame is visible jank

EMBEDDED

Publishing a reading over MQTT

```
func main() {
    led := machine.LED
    led.Configure(machine.PinConfig{Mode:
     machine.PinOutput})
    for {
        led.High()
        time.Sleep(200 * time.Millisecond)
        led.Low()
        time.Sleep(800 * time.Millisecond)
    }
}
```

Orange marks embedded topics.

The `[embedded]` frame option switches the accent for that slide only.

Board

ESP32-C3, flashed with `tinygo flash`.

RECAP

Three things to remember

1. Widgets are immutable blueprints. The State object is what persists across builds.
2. setState marks one Element dirty. Only that subtree rebuilds on the next frame.
3. Value equality decides rebuilds. That is the whole reason Equatable exists.

FURTHER READING

docs.flutter.dev/ui/interactivity · DevTools guide

Questions?

dinu_stefan.rusu@upb.ro · Microsoft Teams: course channel